

Cosmic Shore

Re-run on 4 October against `bleeding-edge` , thirteen days and 315 commits after Revision 5. **The progression blocker stayed closed and the game grew again.** Two things this run corrects are not about the game at all — they are about the machinery that is supposed to prove it works: **the weekly Windows player build retains no player, and it has been failing since 24 September.**

AUDITED	PREVIOUS	BRANCH	WINDOW
4 Oct 2026	Rev 5 · 21 Sep	bleeding-edge 8c4e853a9	315 commits, 13 days

NOT COVERED
Off-machine work; editor runtime

DAYS PAST THE PLAN'S OWN WINDOW 23 Six weeks from 31 July ended 11 September. No week-one gate has been cleared.	DAYS SINCE A WINDOWS PLAYER WAS BUILT 10 Two scheduled runs failed on a static gate, which cancels the build job. The last green run was 24 September — and it kept no player.	LIVE ARCADE CARDS 25 Ten on the day the QA list was scanned. Four arrived in the last nine days.	SUBMITTED QA VERDICTS, ALL TIME 1 Dated 14 August. Unchanged across four revisions of this audit.
---	---	---	--

01 WHAT MOVED, MEASURED WITH THE SAME RULERS

Every figure below was re-measured on 4 October with the predicate the earlier revisions used, not carried forward from them. Where a number looks like it did not move, that is the finding.

GameModes members	55	59	Enum members in <code>GameModes.cs</code> Waystation 58, Dustup 59, Tapestry 60, Sirocco 61
Enabled build scenes	33	36	<code>enabled: 1</code> in <code>EditorBuildSettings.asset</code>
Arcade roster cards	21	25	Entries in <code>ArcadeGames.asset</code> 10 at the QA backlog's 13 Aug scan
Arena roster cards	4	4	Entries in <code>ArenaGames.asset</code>
Vessel classes	12	13	<code>VesselClassType</code> members Butterfly = 13
Ability slots named	29 / 32	33 / 36	<code>element_ability_table.py</code> 3 open: Rhino Charge, Rhino Space, Serpent Mass
Level-5 upgrades wired	23 / 32	29 / 36	Same tool +6 in nine days
QA backlog items	68	68	<code>###</code> QA- headings — byte-identical file
... never run	62	62	Items marked "never run"
QA scan date	13 Aug	13 Aug	The file's own header
Submitted verdicts	1	1	Files in <code>Docs/QA/RESULTS/</code>
Definition of Done	2 / 8	2 / 8	Section 5 below

THREE THINGS GENUINELY LANDED, AND ONE OF THEM IS THE BEST NEWS ON THIS BOARD

A thirteenth vessel arrived complete. Butterfly ships 4/4 abilities named, 4/4 scaling wired and 4/4 level-5 upgrades — the first vessel in the fleet's history to arrive fully mapped rather than be filled in later. The fleet gained six level-5 gates in nine days against twenty-three in the whole year before it.

iOS works end to end. An unsigned `.ipa`, built from a Windows-exported Xcode project on a *free* GitHub-hosted Mac, was sideloaded with a free Apple ID and played on an iPhone on 2 and 4 October. That is a second platform

reached at zero marginal cost, on a test bundle id that deliberately keeps the production identifier clean.

A human verification loop exists. Docs/BugHunt/ documents a Bug_Hunt branch whose stated workflow is: reproduce on bleeding-edge with evidence, fix one item per commit, run the four textual gates, push, **and then a named human pulls it, compiles in the Editor and retests the same repro — nothing is “fixed” until that passes.** Eight problem classes are written up in its playbook and a fix log records each shipped fix with its before-picture. This is the first recurring verification loop the project has ever had, and it is worth more to the Steam date than any single feature in this report.

02 THE BUILD THAT PROVES NOTHING

Revision 5 corrected Revision 4 on this exact row. It recorded H6 as done, on the grounds that *“a weekly green IL2CPP Windows player comes off CI”*, and concluded that R1's remaining cost was *“purely launch it and use it, on a machine, by a person.”* The mechanism is real. The player is not, and the run is not green.

It keeps the manifest and throws the player away

unity-ci.yml builds the player and then runs three upload-artifact steps. They publish the editor log, the edit-mode results, and a single text file, build_manifest.txt . There is no step that uploads the build output.

Measured on the last two runs that went green — 24 September and 17 September — the only artifact either produced is build-manifest-il2cpp , 0.0 MB. So CI proves the player *compiles*, which is genuinely valuable and is what caught the IL1005 linker failure. It hands nobody a player to launch.

The consequence reaches straight into the QA list: QA-BUILD-WINDOWS-PLAYER 's first step offers *“or take the one CI produced from this branch”*. That option does not exist. Whoever runs R1 must install Unity, import the project and build locally.

And it has not run since 24 September

Both scheduled runs since — 29 September and 1 October — **failed**, and neither failed in the build. `check_console_logging.py` failed first, in a separate job, and the `unity` job was **cancelled**. The player build was never attempted.

The cause was one raw `Debug.Log` at `TrainingSessionRunner.cs:710`, on the AI-training branch. It is already fixed; the gate passes on today's head, so the next Thursday run should build. But the shape of it is structural and will recur: **a static gate failing cancels the build job, so a single logging violation on a shared branch costs the week's player build**. Revision 5's “7 of the last 10 were green” is now 3 of the last 6.

<code>upload-artifact</code> × 3 Editor log, edit-mode results, <code>build_manifest.txt</code>. No build output.	<code>build-manifest-il2cpp</code> 0.0 MB — the only artifact on either recent green run.
<code>ios-unsigned-ipa.yml</code> Uploads <code>CosmicShore-dev.ipa</code>, 163 MB, downloadable from the run page.	

THE ASYMMETRY IS THE ARGUMENT

The iOS workflow, added on 2 October as an explicit stopgap, hands back a downloadable installable artifact. The Windows workflow, which has existed far longer and targets the **launch platform**, hands back a text file. **The platform the business depends on is the one with nothing to download.**

Closing it is one `upload-artifact` step — new row **R18**. It is worth doing even though R1 can build locally, because it changes *who* can run the tests: today R1, R3 and H10 all require a Unity install and a long import, which confines them to engineering. An artifact puts them within reach of QA, the artist, and any playtester with a Windows machine.

03 WHAT THE PROGRESSION GATE DOES WHEN YOU TURN IT OFF

Revision 5 closed the blocker three audits had carried: `Menu_Main` now runs a live, configured `GameModeProgressionService`. It also recorded that the master unlock ships **ON**, so the quest graph does not run and nothing is locked. Nobody had looked at the other state. This run did, against the shipped assets.

- 1

`GameModeProgressionService.IsGameModeUnlocked` has exactly **three** ways to return true: the master gate, membership of `ProgressionConfig.alwaysUnlockedModes`, and being the **first** quest in the chain. Everything else falls through to saved progression.
- 2

`alwaysUnlockedModes` holds **22** ids — the four new modes were correctly added to it. The quest chain covers four more: **Scurry** → **Hex Race** → **Joust** → **Maelstrom**, then the Vessel Hangar.
- 3

Cross-referencing that against every roster a player can actually see leaves **five cards in neither set**, and nothing writes saved progression for them. They are locked **permanently**, not slowly.

<code>Scarab Scramble · id 43</code> On the Arcade screen. No quest, not always-unlocked.	<code>Astro League · id 37</code> On the Arena screen.
<code>Brood Rush · id 38</code> On the Arena screen.	<code>Multiplayer Freestyle · id 28</code> On the all-games roster.
<code>Online Duel for the Cell · id 29</code> On the all-games roster.	

TWO MORE FACTS THE SAME MEASUREMENT SETTLED

The free first game is **Scurry**, not **Skim Race**. `firstQuestAlwaysUnlocked` keys on `questList.Quests[0]` — the list order, not the `Order` field — and the list runs **Scurry**, **Hex Race**, **Joust**, **Maelstrom**. Their `Order` fields are 0, 1, 2, 5, so the two agree; it is simply not the mode an outside reader would guess, and “Skim Race” is displayed to players as **HEX RACE**.

An unlocked but unplayed mode caps at intensity 3 (`defaultMaxIntensity`), with **Maelstrom** exempt via `fullIntensityModes`. So with the gate off, a

brand-new player gets **one** game, at intensity 1, and nothing else — and loses it on quitting, because `ProgressionBackendGate.CloudEnabled` is deliberately `false` and progression lives only in a local mirror that resets.

This is a **design decision, not a defect**: add the five to `alwaysUnlockedModes`, give them quests, or accept that they are hidden until the chain completes. It is one asset edit either way, and it has to be answered before the gate can ever ship off — so it blocks Definition of Done #4 the rest of the way. New row **R19**.

04 THE GAP THAT DID NOT CLOSE

The numbers in section 1 that did not move are the ones that decide the Steam date.

The QA list describes a game with ten arcade modes

`Docs/QA/QA_BACKLOG.md` on `bleeding-edge` is byte-identical to the file Revision 5 measured: 68 items, 62 of them never run, generated **13 August**. Measured from that scan commit to today's head: **2,936 commits on the mainline, 1,886 of them merges**. On the day it was written the Arcade roster held 10 cards. It holds 25 now.

Two items were added by hand in this pass — the two states of the progression gate, which did not exist as testable surface until the September merge. The rescan itself is **R2** and is untouched. The backlog's header now says so explicitly, with the commit count, so nobody reads the two new rows as evidence the list is current.

One verdict, still dated 14 August

Four revisions of this audit have now reported the same single submitted QA result. The owner's play report of 21 September — fresh builds installed and running, multiplayer included — remains the strongest evidence the project has that the game works, and it is still **not recorded** as a verdict, so nothing downstream can absorb it. R1 stays 🟡.

R1 has been top of this board since Revision 3. It is now **harder** than Revision 5 recorded it, because the artifact that revision implied does not exist.

05 DEFINITION OF DONE

Unchanged at 2 of 8. One criterion moved in its description without moving in status.

1	Steam page live with Playtest signup. The app record does not exist; R6 has no runbook. Nothing in this window touched it.	BLOCKED
2	Playtest build approved, waves granted, invites on. Still nowhere to upload to — and now, no retained build to upload. The weekly run keeps a manifest.	BLOCKED
3	Fresh install → four-player Tournament, zero P0/P1. Unproven, and the compile proof behind it is 10 days stale.	UNPROVEN
4	Quest chain completes, unlocks persist. Service wired and still wired. Persistence off by flag, and R19 now shows five cards the chain can never reach.	MOVED
5	Load time meets target in every mode. No target published; the matrix grew again, to 100 cells.	NO TARGET
6	Telemetry cohorted by wave. Client half met. Dashboards still to build.	CLIENT MET
7	Zero public review surface. Structural.	MET
8	Gate published, exit checklist signed. Measurable; publishing and signing are still H12.	HALF

WHAT THIS REVISION ASKS FOR, IN ORDER

1. Add the **upload-artifact** step (R18). One step, one afternoon, and it changes who can run three of the human items on the board.
2. Record R1 as a verdict. The substance was reported on 21 September. Copy Docs/QA/RESULTS/TEMPLATE.md , fill it in, press Submit. Until that file exists,

nothing behind it can move and this audit will report one verdict for a fifth time.

3. Run the two new gate items. They are at the top of P0 and take under an hour together. The second one ends by restoring the shipped default; do not skip that step.

4. Answer R19. One asset edit, and a design call that cannot be deferred past the moment the gate ships off.

5. Rescan the QA backlog (R2). It has described a smaller game for seven weeks, and every estimate drawn from it is sized against that smaller game.

Measured 4 Oct 2026 against bleeding-edge 8c4e853a9. Nothing in this report was opened in Unity. Every figure carries the predicate that produced it; where a count differs from an earlier revision, the ruler was re-run on both heads rather than carried forward.